Week 3: Modelling

Data Preparation

1. Importing the necessary libraries

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
```

2. Importing the dataset

In [2]:
```
df = pd.read_csv("student-por.csv", sep=';', encoding='utf-8')
df
```

Out[2]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	healt
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	.
644	MS	F	19	R	GT3	T	2	3	services	other	...	5	4	2	1	2	
645	MS	F	18	U	LE3	T	3	1	teacher	services	...	4	3	4	1	1	
646	MS	F	18	U	GT3	T	1	1	other	other	...	1	1	1	1	1	
647	MS	M	17	U	LE3	T	3	1	services	services	...	2	4	5	3	4	
648	MS	M	18	R	LE3	T	3	2	services	other	...	4	4	1	3	4	

649 rows × 33 columns

3. Data preparation

In [3]:

```
#binary encoding

df['sex'] = df['sex'].map({'F': 0, 'M': 1})                #Female - 0, Male - 1
df['school'] = df['school'].map({'GP': 0, 'MS': 1})        #Gabriel Pereira - 0, Mousinho da Silveira - 1
df['address'] = df['address'].map({'U': 0, 'R': 1})        #urban - 0, rural - 1
df['famsize'] = df['famsize'].map({'LE3': 0, 'GT3':1})     #less than 3 - 0, greater than 3 - 1
df['Pstatus'] = df['Pstatus'].map({'T': 0, 'A': 1})        #living together - 0, apart - 1
df['schoolsup'] = df['schoolsup'].map({'yes': 1, 'no': 0})
df['famsup'] = df['famsup'].map({'yes': 1, 'no': 0})
df['paid'] = df['paid'].map({'yes': 1, 'no': 0})
df['activities'] = df['activities'].map({'yes': 1, 'no': 0})
df['nursery'] = df['nursery'].map({'yes': 1, 'no': 0})
df['higher'] = df['higher'].map({'yes': 1, 'no': 0})
```

```
df['internet'] = df['internet'].map({'yes': 1, 'no': 0})
df['romantic'] = df['romantic'].map({'yes': 1, 'no': 0})
```

In [4]:
```
df = pd.get_dummies(df, columns=['Mjob', 'Fjob', 'reason', 'guardian'], drop_first=True).astype(int)
```

In [5]:
```
df['G_avg'] = (df['G1'] + df['G2']) / 2
df['G_improvement'] = (df['G2'] > df['G1']).astype(int)
```

In [6]:
```
X = df[['studytime', 'absences', 'health', 'G1', 'G2', 'G_avg', 'G_improvement']]  #features
y = df['G3'] #target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state = 42) #splitting data

numeric_features = ['studytime', 'absences', 'health', 'G1', 'G2', 'G_avg', 'G_improvement'] #list of numeric featu

scaler = StandardScaler() #initialization and declaring with variable

X_train[numeric_features] = scaler.fit_transform(X_train[numeric_features])
X_test[numeric_features] = scaler.transform(X_test[numeric_features]) #transforming both features
```

Modelling

1. Linear Regression

In this section, we implement Linear Regression as our baseline model. This model is chosen because it is the simplest form of regression and provides a benchmark against which more complex models can be compared. Linear Regression assumes a linear relationship between the input features and the target variable. While this assumption may not fully capture complex patterns in the data, it serves as a useful starting point for evaluating model performance.

In [7]:
```
from sklearn.linear_model import LinearRegression #importing LinearRegression model

linreg = LinearRegression()
linreg.fit(X_train, y_train)  #training LinearRegression model
```

Out[7]:

```
▾ LinearRegression
▶ Parameters
```

In [8]:

```
#Predicting with the trained model

y_pred_linreg = linreg.predict(X_test)
```

By using fit method we have trained Basic Linear regression model, and by using predict method we have predicted our target variables with feature X_test, furthermore we will compare y_pred with our y_test and calculate errors.

In [9]:

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import cross_val_score  #importing necessary metrics

rmse_linreg = np.sqrt(mean_squared_error(y_test, y_pred_linreg)) #calculating mean squared error

mae_linreg = mean_absolute_error(y_test, y_pred_linreg) #calculating mean absolute error

r2_linreg = r2_score(y_test, y_pred_linreg) #calculating r-squared score

#printing results of Linear Regression
print("Linear Regression: ")
print(f"RMSE: {rmse_linreg}")
print(f"MAE: {mae_linreg}")
print(f"R-squared: {r2_linreg}")
```

```
Linear Regression:
RMSE: 1.1656023612651103
MAE: 0.7361253989200607
R-squared: 0.8606779701250156
```

The Linear Regression model demonstrates strong predictive power with an R^2 score of 0.86, meaning it captures approximately 86% of the underlying data patterns.The error metrics (MAE: 0.736, RMSE: 1.166) suggest that the model is generally accurate. The gap between MAE and RMSE indicates that while most predictions are close to the actual values, there are a few larger errors or outliers that pull the RMSE upward. Overall, this is a solid baseline for a regression task.

In [10]:
```
cv_scores = cross_val_score(linreg, X_train, y_train, cv=5, scoring='r2') #starting to use cross_val_score

cv_mean_linreg = cv_scores.mean() #counting mean to avoid accidentness
cv_std_linreg = cv_scores.std()  #counting standard deviation to see accurateness of model

print("Cross-Validation Results (R2):")
print(f"Mean: {cv_mean_linreg}")
print(f"Std: {cv_std_linreg}")
```

```
Cross-Validation Results (R2):
Mean: 0.8462697969114593
Std: 0.04730737300425364
```

The cross-validation results confirm that your model generalizes well to unseen data. With a mean R^2 of 0.846, the performance is consistent with initial test, proving that the model is not overfitted to a specific data split. The standard deviation of 0.047 indicates a stable model with predictable performance across different subsets of dataset.

In [11]:
```
linreg_results = {
    "Model": "Linear Regression",
    "RMSE": rmse_linreg,
    "MAE": mae_linreg,
    "R2": r2_linreg,
    "CV Mean (R2)": cv_mean_linreg,
    "CV Std": cv_std_linreg
}
# saving results in a dictionary to create a table with all other models, to see properly difference.
```

The Linear Regression model is highly effective for this dataset, achieving a strong balance between accuracy R^2: 0.86 and stability. The minimal gap between the test score and cross-validation mean confirms that the model generalizes well and is ready for practical use or as a solid baseline for more complex algorithms.

---

2. K-Nearest Neighbors

In this section, we implement the K-Nearest Neighbors (KNN) model as an alternative approach. This model is chosen because it is simple and can capture more complex patterns compared to Linear Regression.

KNN makes predictions based on the values of the nearest data points. It assumes that similar inputs will have similar outputs. Unlike Linear Regression, it does not assume a linear relationship between variables.

In [12]:
```
for k in range(1, 21):
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train, y_train)
    print(k, knn.score(X_test, y_test))
```

```
1 0.583505254726827
2 0.7209564088250284
3 0.7650205361790032
4 0.7883015582146065
5 0.8037741802383438
6 0.8140804969674414
7 0.8097344080711336
8 0.8099323666294507
9 0.8138492083825126
10 0.8163321278609742
11 0.8058339673613795
12 0.7999311134573325
13 0.8020165679324438
14 0.8041442797356315
15 0.802813577879717
16 0.8011629534173927
17 0.8023020152112745
18 0.8030151641411076
19 0.8034231681303489
20 0.8012894104512026
```

The best performance was achieved with k = 10, although performance remains stable for values between 5 and 10, indicating that the model is not highly sensitive to this hyperparameter.

In [13]:
```
knn = KNeighborsRegressor(n_neighbors=10)  # The model will look at the 10 closest data points to make each predict

knn.fit(X_train, y_train)  # Train the model using the training data

y_pred_knn = knn.predict(X_test)   # Predict target values for the test set
```

In [14]:
```
rmse_knn = np.sqrt(mean_squared_error(y_test, y_pred_knn))
mae_knn = mean_absolute_error(y_test, y_pred_knn)
r2_knn = r2_score(y_test, y_pred_knn)

print("KNN Regression: ")
print(f"RMSE: {rmse_knn}")
print(f"MAE: {mae_knn}")
print(f"R-squared: {r2_knn}")
```

```
KNN Regression:
RMSE: 1.3383112205600471
MAE: 0.8661538461538463
R-squared: 0.8163321278609742
```

The K-Nearest Neighbors model achieved an RMSE of 1.34, an MAE of 0.87, and an R^2 score of 0.82. This indicates that the model explains about 82% of the variance in the target variable, which is a reasonably strong performance.

However, compared to the Linear Regression model, KNN shows slightly higher prediction errors (RMSE and MAE) and a lower R^2 score. This suggests that, for this dataset, Linear Regression captures the underlying relationship slightly better than KNN.

In [15]:
```
cv_scores_knn = cross_val_score(knn, X_train, y_train, cv=5, scoring='r2')

cv_mean_knn = cv_scores_knn.mean()
cv_std_knn = cv_scores_knn.std()

print("Cross-Validation Results (KNN):")
print("Mean:", cv_mean_knn)
print("Std:", cv_std_knn)
```

```
Cross-Validation Results (KNN):
Mean: 0.8014321136831081
Std: 0.06297518429734268
```

The cross-validation results show that the KNN model performs consistently on unseen data. The mean R^2 of 0.801 is close to the test score, which means the model generalizes well and is not overfitted to a single split. The standard deviation of 0.063 shows some variation across folds, but overall the model remains fairly stable.

In [16]:
```
knn_results = {
    "Model": "KNN Regression",
    "RMSE": rmse_knn,
    "MAE": mae_knn,
    "R2": r2_knn,
    "CV Mean (R2)": cv_mean_knn,
    "CV Std": cv_std_knn
}
# saving results in a dictionary to create a table with all other models, to see properly difference.
```

The KNN model performs well on this dataset, with a good balance of accuracy ($R^2 \approx 0.82$) and consistency. The close values between the test score and cross-validation mean show that the model generalizes reasonably well and is not heavily overfitted. However, its performance is slightly lower than Linear Regression, making it a good alternative but not the strongest model for this dataset.

---

3. Decision Tree

In this section, we implement the Decision Tree model as another approach. This model is selected because it can break the data into smaller parts and model complex relationships without requiring a predefined formula.

Decision Tree works by creating a sequence of splits based on feature values, forming a tree-like structure of decisions. Each split helps to better separate the data and improve prediction accuracy. Unlike Linear Regression, it does not rely on a linear relationship and can adapt to more irregular patterns in the dataset.

In [17]:
```
dt = DecisionTreeRegressor(max_depth=5, random_state=42)
# limiting tree depth to reduce overfitting and improve generalization
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

In [18]:
```
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
mae_dt = mean_absolute_error(y_test, y_pred_dt)
r2_dt = r2_score(y_test, y_pred_dt)

print("Decision Tree Regression:")
```

```
print(f"RMSE: {rmse_dt}")
print(f"MAE: {mae_dt}")
print(f"R-squared: {r2_dt}")
```

```
Decision Tree Regression:
RMSE: 1.208829463355163
MAE: 0.7318153554114807
R-squared: 0.8501526628726458
```

The Decision Tree model was tuned by setting max_depth = 5 to limit model complexity and reduce overfitting. This helps the model generalize better to unseen data.

With this configuration, the model achieves an R^2 score of 0.85, showing strong performance that is comparable to Linear Regression and KNN. This demonstrates that controlling model complexity is important for improving Decision Tree performance.

In [19]:
```
cv_scores_dt = cross_val_score(dt, X_train, y_train, cv=5, scoring='r2')

cv_mean_dt = cv_scores_dt.mean()
cv_std_dt = cv_scores_dt.std()

print("Cross-Validation Results (Decision Tree):")
print("Mean:", cv_mean_dt)
print("Std:", cv_std_dt)
```

```
Cross-Validation Results (Decision Tree):
Mean: 0.8469478221852803
Std: 0.08338068682174578
```

The cross-validation results for the Decision Tree model show a mean R^2 of 0.847, which is very close to the test score, indicating good generalization. The standard deviation of 0.083 suggests some variability across different data splits, but overall the model remains reasonably stable after tuning.

In [20]:
```
dt_results = {
    "Model": "Decision Tree",
    "RMSE": rmse_dt,
    "MAE": mae_dt,
    "R2": r2_dt,
    "CV Mean (R2)": cv_mean_dt,
```

```
    "CV Std": cv_std_dt
}
```

The Decision Tree model performs well after tuning, achieving an R^2 of approximately 0.85. The close values between the test score and cross-validation mean indicate that the model generalizes well and is not significantly overfitted. Overall, the model shows stable and reliable performance across different data splits.

A comparison table

In [21]:
```
results_df = pd.DataFrame([
    linreg_results,
    knn_results,
    dt_results
])

results_df
```

Out[21]:

	Model	RMSE	MAE	R2	CV Mean (R2)	CV Std
0	Linear Regression	1.165602	0.736125	0.860678	0.846270	0.047307
1	KNN Regression	1.338311	0.866154	0.816332	0.801432	0.062975
2	Decision Tree	1.208829	0.731815	0.850153	0.846948	0.083381

Linear Regression shows the most balanced performance overall, achieving the highest R^2 score (0.86) and the lowest prediction error (1.166) among the models. This suggests that the relationship between the features and the target variable is largely linear, which favors simpler models.

KNN performs slightly worse, likely because it relies heavily on local similarity. If the feature space is not uniformly distributed or contains noise, distance-based methods can become less effective.

Decision Tree performs close to Linear Regression after tuning, but shows higher variability (as seen in CV standard deviation). This indicates that while it can capture non-linear patterns, it is more sensitive to how the data is split and may not generalize as smoothly.

Overall, the models agree on the general structure of the data, but differ in how they handle noise and local variations: Linear Regression provides stability, KNN focuses on local patterns, and Decision Tree is more flexible, but less stable.

Progress Log

We started with Linear Regression as a baseline model to see how well a simple approach works on this dataset. It gave a strong starting point and showed that the data has a fairly clear structure.

After that, we tried KNN because it can capture more complex patterns. The main challenge was choosing the value of k. We tested different values and saw that very small or very large values made the model worse, while mid-range values worked better.

Then we used a Decision Tree model to try a different approach based on rules and splits. At first, the model was overfitting the data, so we limited its complexity using max_depth. This improved the results and made it more stable.

One important part of the project was making sure all models were evaluated in a consistent way using the same data splits and cross-validation. Another thing we noticed is that even when the scores are close, the models behave differently in how they make predictions.

Overall, we learned that simple models can already work quite well, and more complex models need careful tuning to perform properly.